

Qwen3-Coder-Next Tier 3

Part 8 Detailed Implementation Specification

Failure Recovery

Internal engineering specification for building the recovery subsystem from scratch.

Document role	What this part delivers	Depends on	Enables
Engineering spec, not roadmap or architecture	Failure diagnosis, strategy selection, safe retry, escalation, and recovery records	Architecture PDF, Master Roadmap, Parts 1-7	Parts 9-15, especially repository intelligence, graph reasoning, benchmark harness, and deployment approval

Design principle: never let a failure become an uncontrolled loop. Every failure must end in a bounded recovery decision: retry, reroute, escalate, or abort safely.

1. Purpose

This part exists to prevent the system from treating every failure like a dead end. In a multi-agent coding workflow, failures are normal: tools time out, tests fail, context is incomplete, permissions are missing, external services are flaky, and the model occasionally makes a bad plan while sounding confident about it. The recovery layer converts those failures into controlled outcomes instead of letting the orchestrator panic or repeat the same mistake.

The subsystem solves three problems at once. First, it classifies the failure so the system knows whether the issue is transient, semantic, environmental, or unrecoverable. Second, it selects a bounded response strategy instead of blindly retrying. Third, it preserves evidence and state so the next action is informed, auditable, and safe.

This part is a dependency for the rest of the stack. Testing and review depend on it because they will generate frequent failure signals. Repository intelligence, knowledge graph navigation, and context compression will later improve diagnosis quality, but they all assume the recovery shell already exists. Deployment and human approval also depend on it because any unsafe terminal failure must be escalated cleanly, not hidden.

Goal: turn failure from a system crash into a managed control flow.

2. Scope

Included in Part 8	Excluded from Part 8
Failure event ingestion, normalization, classification, strategy selection, retry budgets, checkpoint and rollback handling, safe abort, human escalation, recovery records, and recovery metrics.	Planning, research, code generation, test execution itself, repository indexing, knowledge graph construction, and deployment execution beyond the handoff boundary.

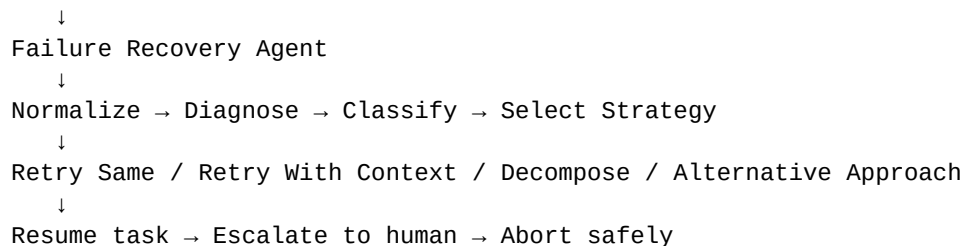
Included work is limited to the control plane around failures. It does not implement the failing work itself. The recovery agent listens, decides, and routes. It does not replace the coding agent, testing agent, or deployment agent.

Excluded work belongs to later parts or other subsystems. Repository intelligence will supply richer diagnostics later. Knowledge graph queries will improve impact analysis later. Benchmark harnessing will measure recovery quality later. None of that changes the basic need for a safe recovery loop now.

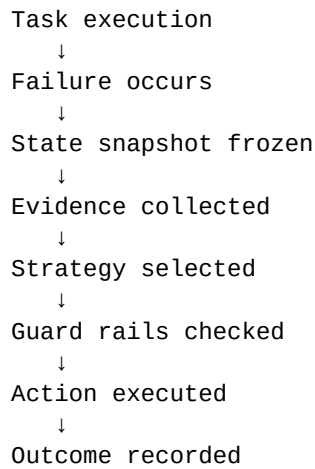
3. System Overview

The recovery path is a separate control flow that starts only after another component reports a failure. The agent normalizes the failure, analyzes evidence, chooses a strategy, and then either resumes the task, redirects it, or ends it safely.

```
User / Orchestrator
↓
Coding Agent / Testing Agent / Review Agent / Deployment Agent
↓
Failure Event
```

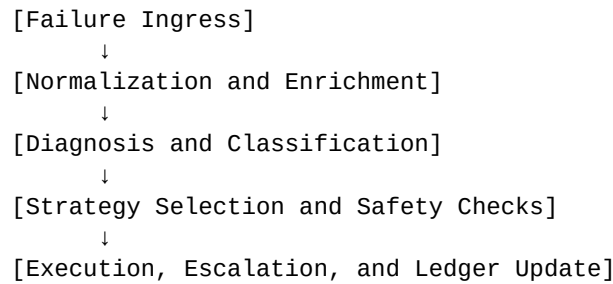


The recovery flow always returns a terminal decision. A failure may produce a new attempt, but it must never disappear into an unbounded loop. The recovery subsystem therefore acts like a guardrail around the whole system.



4. Internal Architecture

The recovery subsystem is easiest to understand as a small pipeline with four layers. Each layer has one job and one output. That keeps diagnosis deterministic and makes the unit tests possible.



The main internal components are:

Component	Responsibility	Inputs	Outputs
Failure Ingress Adapter	Receives error reports from agents, tools, and orchestration boundaries. Converts arbitrary exceptions into a normalized Failure Event.	Raw exceptions, stack traces, tool errors, agent signals	Normalized failure event
Evidence Collector	Captures the last known state needed to understand what	Task state, logs, command history, worktree ref, memory	Evidence bundle

	failed.	refs	
Diagnosis Engine	Determines failure class, root cause candidate, and confidence.	Failure event, evidence bundle, policy rules	Diagnosis report
Strategy Selector	Chooses the best bounded response based on diagnosis and budget.	Diagnosis report, retry budgets, safety policy	Recovery plan
Recovery Executor	Applies the plan by retrying, rerouting, or aborting.	Recovery plan, task state, worktree manager, orchestrator hooks	Recovery outcome
Ledger and Metrics Writer	Persists what happened so the task can be replayed and the system can learn later.	Outcome, evidence, timestamps, counts	Recovery record

Required abstractions are deliberately narrow. The recovery system must be able to ask four questions only: What failed? Why did it fail? What is the safest next move? What evidence do we keep?

Important rule: recovery should never mutate production branches directly. Any retry that edits code must happen in an isolated worktree or equivalent sandbox.

5. Project Structure

A clean directory layout prevents the recovery logic from becoming a junk drawer. The following structure separates contracts, strategies, execution, persistence, and tests.

```
recovery/
  __init__.py
  contracts.py
  config.py
  events.py
  evidence.py
  classifier.py
  diagnosis.py
  policy.py
  budgets.py
  executor.py
  router.py
  ledger.py
  metrics.py
  checkpoints.py
  strategies/
    __init__.py
    retry_same.py
    retry_with_context.py
    decompose.py
    alternative.py
    escalate.py
```

```

    abort.py
    integrations/
        orchestrator.py
        memory.py
        worktree.py
        logs.py
    tests/
        unit/
        integration/
        chaos/
        scenarios/
    configs/
        recovery.yaml
        strategy_rules.yaml
        retry_budgets.yaml
    state/
        recovery_state.py
        recovery_history.py
        failure_fingerprints.py
    artifacts/
        recovery_records/
        diagnostics/
        escalation_packets/

```

Path	Purpose
recovery/contracts.py	Canonical data contracts for failure events, diagnosis, recovery plans, and outcomes.
recovery/classifier.py	Failure taxonomy and confidence scoring.
recovery/diagnosis.py	Root-cause analysis orchestration.
recovery/strategies/	Bounded action implementations for each recovery path.
recovery/executor.py	Runs the selected strategy and manages side effects.
recovery/checkpoints.py	Snapshot and restore hooks for state and worktree handling.
recovery/ledger.py	Persistence for recovery history and replayability.
state/	Shared runtime state and event-sourced recovery history.
tests/	Focused verification for every strategy and failure type.

6. Data Contracts

All recovery logic depends on explicit contracts. The system should not pass raw exceptions around as if they were enough. They are not. Evidence, context, and budgets must be carried alongside the error.

Contract	Meaning
FailureEvent	Normalized input from any failing component. Contains task identity, source agent, failure type, severity, raw message, stack trace reference, tool reference, timestamp, retry

	count, and worktree reference.
EvidenceBundle	Structured snapshot of the state at the moment of failure. Includes recent actions, logs, command output, memory references, and relevant file or module anchors.
DiagnosisReport	Classifier output with failure category, probable root cause, confidence, evidence summary, and recommended recovery direction.
RecoveryPlan	Strategy choice plus operational details such as retry budget consumption, backoff, additional context request, rollback steps, or escalation packet.
RecoveryOutcome	Terminal result of execution. Records whether the task resumed, was rerouted, escalated, or aborted safely, with outcome notes and references.
RecoveryRecord	Persisted ledger entry combining event, diagnosis, plan, outcome, and timestamps.

Structured example for a failure event:

```
{
  task_id: "task-042",
  source_agent: "Testing Agent",
  failure_type: "timeout",
  severity: "high",
  message: "pytest exceeded 180s",
  stack_trace_ref: "logs/run-18.txt#L45-L82",
  worktree_ref: "wt-9f31",
  retry_count: 1,
  budget_remaining: 2
}
```

Structured example for a recovery plan:

```
{
  strategy: "retry_with_context",
  reason: "transient tool timeout",
  context_delta: "restore longer timeout and attach recent test log",
  backoff_seconds: 15,
  preserve_worktree: true,
  max_attempts_after_plan: 1
}
```

Contract rule: every recovery decision must be serializable and replayable. If it cannot be logged, it is not a valid recovery decision.

7. State Management

Recovery state is split into two layers. Ephemeral state lives in the orchestrator while the task is active. Persistent state lives in a recovery ledger and, when appropriate, in the project memory layer from Part 5. That separation keeps the runtime responsive without losing history.

State layer	What it stores	Ownership
Ephemeral task recovery state	Current failure, current plan, retry count, guardrail status, and last evidence bundle.	Owned by the recovery runtime during the live task.
Persistent recovery ledger	All failure events, diagnoses, plans, outcomes, and timestamps.	Owned by the recovery subsystem and written on every terminal decision.
Failure fingerprints	Normalized signatures that link similar failures across runs.	Used to improve future classification and to support regression analysis.
Recovery memory summary	Compressed notes about repeated failures or especially useful remedies.	Published to project memory for later retrieval.

State changes only through explicit transitions. A task enters recovery when a failure event is accepted. It leaves recovery only when the system resumes execution, escalates to a human, or aborts safely. Every transition must write a record.

Future agents consume this state in two ways. First, they read the live task state to avoid repeating the same step. Second, they query the ledger or memory summary to learn whether a previous remedy already worked.

8. Component Specifications

8.1 Failure Ingress Adapter

Purpose: accept failure signals from agents, tools, and orchestration boundaries.

Inputs: raw exceptions, structured error envelopes, tool exit codes, timeout notices.

Outputs: normalized FailureEvent objects.

Responsibilities: validate source, attach task metadata, preserve the original error payload, reject malformed events.

Failure conditions: missing task id, unsupported source, corrupted payload.

Future integration: later classifiers and graph-aware diagnostics can consume the same normalized event.

8.2 Evidence Collector

Purpose: assemble the minimum evidence needed to explain the failure.

Inputs: logs, recent action history, worktree ref, memory references, agent status.

Outputs: EvidenceBundle.

Responsibilities: fetch recent context, capture relevant artifacts, avoid oversharing irrelevant tokens.

Failure conditions: unavailable logs, expired snapshot, missing worktree ref.

Future integration: later context compression will reduce evidence size without changing semantics.

8.3 Diagnosis Engine

Purpose: classify the failure and identify likely root cause.

Inputs: FailureEvent + EvidenceBundle + policy rules.

Outputs: DiagnosisReport.

Responsibilities: map failure into a controlled taxonomy, score confidence, identify evidence that supports the classification.

Failure conditions: contradictory evidence, unknown failure class, insufficient context.

Future integration: repository intelligence and knowledge graph can improve root-cause scoring.

8.4 Strategy Selector

Purpose: choose the safest bounded response.

Inputs: DiagnosisReport, budgets, runtime policy.

Outputs: RecoveryPlan.

Responsibilities: select `retry_same`, `retry_with_context`, `decompose`, `alternative_approach`, `escalate`, or `abort`.

Failure conditions: policy conflict, exhausted budget, unsafe retry condition.

Future integration: benchmark harness will later measure strategy success by failure class.

8.5 Recovery Executor

Purpose: carry out the selected response.

Inputs: RecoveryPlan, task state, worktree manager, orchestrator hooks.

Outputs: RecoveryOutcome.

Responsibilities: apply backoff, restore checkpoints, restart attempts, route subtasks, or shut down cleanly.

Failure conditions: retry loop violation, worktree conflict, side effect leak.

Future integration: deployment stage will reuse the same safety and escalation primitives.

8.6 Ledger and Metrics Writer

Purpose: make recovery observable and replayable.

Inputs: event, diagnosis, plan, outcome.

Outputs: RecoveryRecord and runtime metrics.

Responsibilities: persist records, emit counters and timers, preserve failure fingerprints.

Failure conditions: storage failure, schema mismatch, telemetry outage.

Future integration: benchmark harness will use these records for success-rate and retry-quality reporting.

9. Implementation Sequence

Step 1 - Define contracts and taxonomy

Start with the event shapes, diagnosis taxonomy, and outcome model. Everything else depends on these names being stable. Expected output: typed schemas, enums, and serialization rules.

Dependencies: none other than the orchestration boundary.

Step 2 - Implement failure ingestion and normalization

Build the adapter that converts raw errors into normalized events. This is first because nothing else is useful without reliable input. Expected output: consistent FailureEvent creation.

Dependencies: contracts from Step 1.

Step 3 - Add evidence capture

Capture logs, recent actions, worktree refs, and memory anchors. This comes next because diagnosis needs context, not guesses. Expected output: EvidenceBundle generation. Dependencies: task state and logging hooks.

Step 4 - Implement deterministic classification

Create the first version of the diagnosis engine using rules and confidence thresholds. Expected output: a stable failure category for the common classes. Dependencies: event and evidence models.

Step 5 - Build the strategy registry

Map diagnosis classes to bounded recovery options and budget rules. Expected output: a policy-backed strategy selector. Dependencies: classifier output and runtime budgets.

Step 6 - Implement retry and escalation execution

Wire the strategy executor to perform retries, rerouting, safe aborts, and human escalation.

Expected output: terminal recovery outcomes. Dependencies: strategy registry and orchestrator callbacks.

Step 7 - Add checkpoint and rollback handling

Before aggressive retries, ensure state can be restored or safely discarded. Expected output: reversible attempts and clean rollback. Dependencies: worktree manager and task snapshot hooks.

Step 8 - Persist ledger and emit metrics

Write recovery records and counters after every terminal decision. This is necessary for auditing and later benchmarking. Expected output: durable recovery history. Dependencies: outcome model and storage.

Step 9 - Add scenario tests and chaos tests

Exercise the full flow against synthetic and real failure cases. Expected output: a verified recovery pipeline with no uncontrolled loops. Dependencies: the full implementation.

10. Validation Strategy

Test type	What it proves
Unit tests	Validate taxonomy mapping, budget logic, strategy selection, and contract validation.
Integration tests	Simulate failures from Coding, Testing, Review, and Deployment agents and verify terminal outcomes.
Regression suite	Replay a curated corpus of known failures to ensure the same class still gets the same treatment.
Chaos tests	Inject random timeouts, permission errors, tool crashes, malformed outputs, and OOM conditions.
Safety tests	Verify no infinite retry, no branch mutation outside the worktree, and no lost evidence.
Replay tests	Re-run archived failure records and confirm the decision trace is reproducible.

Correctness is not just whether the system eventually continues. It is whether the system continues for the right reason, with the right evidence, under the right constraints. Regressions are prevented by replaying the same failure corpus on every meaningful change to strategy, classification, or budgets.

A recovery system that retries forever is not resilient. It is just a slower failure.

11. Deliverables

- [] Recovery contract definitions and taxonomy
- [] Failure ingestion adapter
- [] Evidence collection module
- [] Diagnosis engine
- [] Strategy registry and policy rules
- [] Retry budget configuration
- [] Checkpoint and rollback hooks
- [] Recovery ledger and metrics writer
- [] Escalation packet generator
- [] Scenario and chaos test suite
- [] Recovery documentation and operator notes

12. Acceptance Criteria

Criterion	Verification method	Pass condition
Every supported failure class produces a diagnosis	Run the failure corpus and confirm each event ends in a diagnosis report.	100% coverage of defined failure classes.

Every terminal decision is logged	Inspect the ledger after unit and integration tests.	No recovery path exits without a record.
Retry budgets are enforced	Inject repeated transient failures and count retries.	No path exceeds configured retry limits.
Safe abort preserves evidence	Force unrecoverable failures and inspect artifacts.	Logs, snapshots, and failure fingerprints remain available.
Human escalation packets are actionable	Review the packet content produced for unknown or unsafe failures.	Packet contains task id, cause, evidence refs, and next action.
Recovery decisions are reproducible	Replay the same event with the same config.	Same classification and same strategy choice.
Worktree integrity remains intact	Compare pre- and post-recovery branches or worktrees.	No unauthorized mutation of the main branch.

These checks must be automated. A recovery subsystem that only passes code review is not complete. It must survive the ugly reality of failed tools, incomplete context, and noisy environments.

13. Common Failure Modes

- Retrying the same broken action with no new evidence. This creates loops and burns time.
- Treating all failures as transient. That hides semantic bugs and wastes retries.
- Overfitting the classifier to one tool or one agent. Recovery must be generic across the stack.
- Dropping the original error when normalizing events. Once evidence is lost, diagnosis gets weaker.
- Mutating code outside an isolated worktree. That breaks rollback safety.
- Allowing recovery to continue without a budget cap. This is a direct path to infinite loops.
- Escalating too late. If the system is unsure and the budget is nearly exhausted, it should escalate early.
- Writing recovery records only on success. Failed recoveries are often the most important records.

Scalability traps are mostly about context size and decision latency. If evidence capture becomes expensive, the system will hesitate at the worst possible moment. Keep the default path compact and make the expensive diagnostics optional and bounded.

14. Future Integration

Next part	How it depends on recovery
Part 9 - Repository Intelligence	Improves diagnosis by exposing folder, module, and dependency summaries for better root-cause analysis.
Part 10 - Code Knowledge Graph	Allows recovery to ask what is affected before choosing a retry or rollback path.
Part 11 - Context Compression	Reduces the size of retry context while preserving the evidence that actually matters.

Part 12 - Multi-Agent Coordination	Lets recovery route work back to the most appropriate agent instead of the same agent blindly retrying.
Part 13 - Benchmark Harness	Measures recovery success rate, retry count, escalation rate, and worktree discard rate.
Part 14 - Deployment and Human Approval	Uses the same safe escalation packet and approval gate when deployment fails.
Part 15 - Near-Codex Integrated System	Treats recovery as a first-class control layer across planning, research, coding, review, evaluation, and deployment.

The interface must remain stable even as future parts make the diagnosis smarter. Later modules may provide richer evidence, but they should not force the recovery contract to change every time the system learns something new.

15. Completion Checklist

- [] Contracts defined for FailureEvent, EvidenceBundle, DiagnosisReport, RecoveryPlan, RecoveryOutcome, and RecoveryRecord.
- [] Failure normalization works for agent errors, tool errors, timeouts, permission failures, OOM events, and unknown exceptions.
- [] Evidence collector captures task context, recent actions, logs, and worktree references.
- [] Classifier assigns a bounded failure category with a confidence score.
- [] Strategy selector enforces budgets and chooses only allowed recovery actions.
- [] Executor can retry, reroute, escalate, or abort safely.
- [] Checkpoint and rollback hooks are wired to the worktree and task state.
- [] Every terminal outcome is persisted in the recovery ledger.
- [] Metrics are emitted for retries, outcomes, latency, and escalations.
- [] Unit, integration, chaos, and replay tests all pass.
- [] No recovery path can mutate the main branch directly.
- [] Human escalation packets are complete and actionable.